

c# Programmieren (c) HTL-Leonding

CoffeeSlot-Machine

Lehrziele

- Modellierung einfacher Klassen - Verhaltensmethoden und Properties
- Konstruktor - Defaultkonstruktor und überladener Konstruktor
- Zugriffsmodifizierer

Aufgabenstellung

Ein Kaffeeautomat, wie er bei uns an der Schule steht, ist zu modellieren.

Zur Verwaltung des Zustands des Automaten empfiehlt es sich im Inneren des Objekts (gekapselt) folgende Informationen als private Fields zu speichern:

- Namen der vom Automaten angebotenen Produkte als Stringarray
- Gültige Münzwerte (5 Cent – 200 Cent) als Integerarray
- Inhalt der Kasse / des Münzdepots (wie viele Münzen von jedem Münzwert) als Integerarray
- Zähler welches Produkt wie oft gekauft wurde als Integerarray
- Der Preis wird als Konstante definiert und beträgt für alle Produkte 50 Cent
- Der Münzeinwurf des aktuellen Verkaufsvorgangs muss ebenfalls verwaltet werden
 - Welche Münzwerte wurden wie oft eingeworfen (Integerarray). Bei Storno müssen diese wieder retourniert werden
 - Gesamtsumme des aktuellen Einwurfs (integer)

Folgendes Verhalten muss der Kaffeautomat aufweisen (ist nach außen sichtbar):

Konstruktoren

- Der Defaultkonstruktor initialisiert die Kasse **mit jeweils drei Münzen je Münzwert** und die Produkte mit den drei Standardprodukten **Cappuccino, Mocca, Kakao**
- Ein weiterer Konstruktor bekommt die Vorbelegung der Kasse (Integerarray) und die Produktnamen (Stringarray) übergeben

Properties

- **CoinsInDepot** gibt zurück, wie viele Münzen sich derzeit in der Kasse befinden (nur die **Gesamtanzahl der Münzen als integer!!** Nicht das Integerarray). Aktueller Einwurf wird bis zur Produktauswahl nicht gezählt
- **ProductsAvailable** liefert die Anzahl verfügbarer Produkte
- **Credit** Wert der eingeworfenen Münzen - wie viel wurde schon bezahlt!

Businessmethoden

- **InsertCoin**: Eine Münze wird eingeworfen. Der Wert wird in Cent angegeben. Ungültige Werte (z.B. 17) fallen genau so durch, wie unzulässige Münzen (1 Cent, 2 Cent). Wurden schon zumindest 50 Cent

eingeworfen, fällt die Münze ebenfalls durch. Der Rückgabewert signalisiert die fehlerfreie Übernahme der Münze **SelectProduct**: Der Kunde wählt das Produkt über den Namen aus.

- Falls es nicht existiert oder zu wenig Geld eingeworfen wurde, liefert die Methode false zurück.
- Sonst wird der jeweilige Produktzähler erhöht und das eingeworfene Geld in die Kasse übernommen.
- Weiters wird das Wechselgeld bestimmt und aus der Kassa abgezogen (Wechselgeld mit möglichst hohen Münzwerten).

Im ersten out-Parameter soll die Anzahl der Münzen des Wechselgeldes als Intarray zurückgeliefert werden.

Im zweiten out-Parameter soll zurückgeliefert werden, wie viel Cent nicht zurückgegeben werden können (wenn sich zu wenig Münzen im Automaten befinden).

- **CancelOrder**: Die aktuelle Bestellung wird abgebrochen.
Die Anzahl der eingeworfenen Münzen wird je Münzwert in einem Array zurückgegeben (Index: 5 Cent => 0, 200 Cent => 5)
- **EmptyDepot**: Kassa leeren und Summe der Centbeträge zurückgeben.
- **GetCounterForProduct** liefert die Anzahl der Bestellungen für das Produkt über einen out- Parameter.
Gibt es den Produktnamen nicht, wird false zurückgegeben
- **GetCounterForCoin** liefert die Anzahl der Münzen in der Kasse für den übergebenen Münzwert über einen out-Parameter.
Gibt es den Münzwert nicht, wird false zurückgegeben

Für alle Elemente (Konstruktoren, Properties, Methoden) gibt es Unittests. Ihre Aufgabe ist es, diese Elemente in der Klasse CoffeeSlotMachine so zu implementieren, dass die Unittests bestanden werden.